

POD-ATTENTION: Unlocking Full Prefill-Decode Overlap for Faster LLM Inference

Aditya K Kamath*
University of Washington
Seattle, USA

Ramya Prabhu
Microsoft Research
Bengaluru, India

Jayashree Mohan
Microsoft Research
Bengaluru, India

Simon Peter
University of Washington
Seattle, USA

Ramachandran Ramjee
Microsoft Research
Bengaluru, India

Ashish Panwar
Microsoft Research
Bengaluru, India

Abstract

Each request in LLM inference goes through two phases: compute-bound *prefill* and memory-bandwidth-bound *decode*. To improve GPU utilization, recent systems use hybrid batching that combines the prefill and decode phases of different requests into the same batch. This approach optimizes linear operations but remains inefficient for attention computation because *existing attention kernels specialize execution independently for the prefill and decode phases*.

In this paper, we present POD-ATTENTION — the first GPU kernel that efficiently computes attention for hybrid batches. POD-ATTENTION aims to maximize the utilization of both compute and memory bandwidth by carefully allocating the GPU’s resources such that prefill and decode operations happen concurrently on the same multiprocessor. POD-ATTENTION speeds up attention computation by up to 59% (mean 28%), enabling higher throughput and lower latency LLM inference compared to the use of independently optimized prefill and decode attention kernels.

CCS Concepts: • **Computing methodologies** → *Machine learning*; • **Computer systems organization** → **Parallel architectures**.

Keywords: Large language models; GPUs; self-attention

ACM Reference Format:

Aditya K Kamath, Ramya Prabhu, Jayashree Mohan, Simon Peter, Ramachandran Ramjee, and Ashish Panwar. 2025. POD-ATTENTION: Unlocking Full Prefill-Decode Overlap for Faster LLM Inference. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS ’25)*, March 30–April 3, 2025, Rotterdam, Netherlands. ACM, New York, NY, USA, 16 pages. <https://doi.org/10.1145/3676641.3715996>

*Work done as an intern at Microsoft Research India.



This work is licensed under a Creative Commons Attribution 4.0 International License.

ASPLOS ’25, March 30–April 3, 2025, Rotterdam, Netherlands

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1079-7/2025/03

<https://doi.org/10.1145/3676641.3715996>

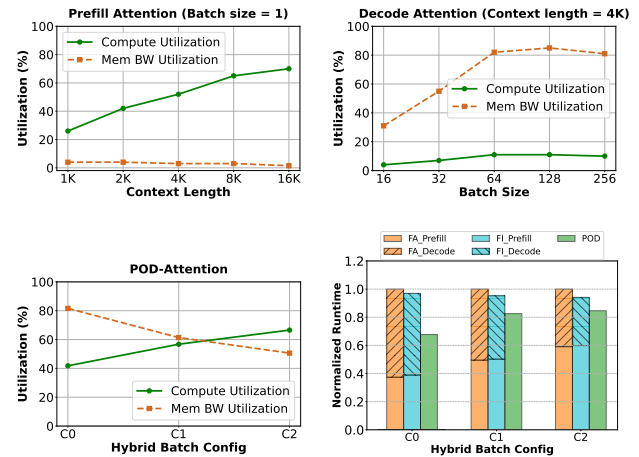


Figure 1. State-of-the-art attention kernels utilize either compute or memory (FA: FlashAttention, FI: FlashInfer). POD-ATTENTION utilizes both compute and memory to accelerate attention computation in hybrid batches (see Table 1 for configurations. Model: Llama-3-8B on 2 A100 GPUs).

1 Introduction

The infrastructure for serving large language models (LLMs) is expanding to meet their growing demands [3, 16]. Large-scale service providers often depend on expensive high-end GPUs to meet peak demand or latency targets [46]. Therefore, optimizing LLM serving systems has become crucial [21, 23, 41, 57, 62, 65, 66]. The overall efficiency of a deployment depends on how well GPU resources are utilized.

From a resource utilization perspective, LLM inference is a challenging workload because different phases require different resources at different times [22–24, 66]. The processing of an LLM request begins with a highly parallel (hence, compute-bound) prefill phase which is then followed by a memory-bound decode phase [24]. Serving LLMs efficiently, therefore, requires both high compute and high memory bandwidth. An ideal system would strive to maximize the utilization of both compute and memory. However, doing so is non-trivial because for a given request, the prefill and decode phases occur at different times.

State-of-the-art LLM serving systems deal with this challenge by combining the inputs of prefill and decode phases of different requests into the same batch [24, 33, 62] — a technique we refer to as *hybrid batching*. Hybrid batching avoids the need to fetch model weights from GPU high-bandwidth memory (HBM) separately for prefill and decode tokens. Instead, it allows the GPU to fetch model weights once and use them to compute over both prefill and decode inputs. Hybrid batching also helps reduce tail latency: to limit the runtime of each iteration, the scheduler can divide long input prompts (prefill inputs) into multiple smaller chunks, then combine ongoing decodes with a new prefill chunk every iteration [23, 33]. As such, use of hybrid batching is common in various LLM serving systems today [23, 33, 41, 62, 66].

While prior work has focused on optimizing the linear operations [23, 33, 62], they do not optimize the attention computation of a hybrid batch. This is reasonable for a system that primarily deals with small context lengths since linear operations dominate run time in this setting [62, 66]. In contrast, as the context length increases, attention computation becomes the primary performance bottleneck (Figure 4).

Some recent works have also tried to optimize attention computation [30, 31, 34, 48], but current solutions address prefill and decode operations separately — maximizing compute utilization for prefills and bandwidth utilization for decodes, as shown in Figure 1. In this paper, we show that such an approach is suboptimal as it leaves critical GPU resources underutilized in different parts of computation. For example, Figure 1 illustrates that memory bandwidth utilization of the prefill attention kernel is often below 5%, while compute utilization of the decode attention kernel is under 10%. The effect of using independently optimized kernels is particularly noticeable with hybrid batching because prefill and decode kernels execute immediately one after the other, leading to periods of high demand of a resource immediately followed by low utilization of the same resource.

To improve the efficiency of hybrid batching, we present POD-ATTENTION — the first GPU kernel, to the best of our knowledge, that efficiently batches the computation of prefill and decode attention. In doing so, we first show (§3) that existing techniques do not provide adequate performance in fusing attention computation due to various limitations such as straggler threads, synchronization barriers and lack of guaranteed SM-level co-location of different Cooperative Thread Arrays (CTAs) on GPU Streaming Multiprocessors (SMs). POD-ATTENTION addresses these issues by fusing the computation in a CTA-parallel manner, introducing SM-aware software-based CTA scheduling within the GPU (§4). Building on state-of-the-art FlashAttention kernels [1], POD-ATTENTION significantly accelerates attention computation by utilizing both compute and memory resources as per the requirement of a given batch of requests (see Figure 1).

Config.	Prefill			Decode		Resource requirement
	BS	CS	CL	BS	CL	
C0	1	1K	12K	80	12K	memory-bound
C1	1	12K	12K	220	12K	balanced
C2	1	16K	16K	250	12K	compute-bound

Table 1. Details of hybrid batches evaluated in Figure 1 (BS: batch size, CS: chunk size, CL: context length).

We also integrate POD-ATTENTION in a state-of-the-art LLM inference scheduler Sarathi-Serve [23]. Our experiments show that POD-ATTENTION computes attention up to 59% faster (mean 28%) than the prefill and decode attention kernels of FlashAttention and FlashInfer. In terms of the end-to-end LLM inference performance, POD-ATTENTION improves throughput by up to 22% while also reducing crucial latency metrics such as time-to-first-token (TTFT), time-between-tokens (TBT) and the end-to-end request execution latency over Sarathi-Serve.

Contributions: We make the following contributions:

- We highlight that independently optimizing prefill and decode attention kernels is suboptimal for hybrid batching based LLM inference.
- We present POD-ATTENTION — a GPU kernel that computes prefill and decode attention concurrently to utilize both compute and memory bandwidth simultaneously.
- We integrate POD-ATTENTION in Sarathi-Serve and show that it enables high throughput and low latency LLM inference compared to the use of independently optimized prefill and decode attention kernels.

2 Background and Motivation

We first discuss why LLM serving systems use hybrid batching and then motivate the need to optimize attention computation. Finally, we provide an overview of GPU execution.

2.1 Large Language Model (LLM) Inference

LLMs process user inputs and outputs as tokens, internally represented as vectors. Each request during inference goes through two phases — prefill and decode [62]. The prefill phase processes the tokens of a user’s prompt in parallel and produces the first output token, whose latency is called time-to-first-token (TTFT). Subsequently, the decode phase generates one output token (per-request) per-iteration autoregressively. The latency taken to generate each output token is called time-between-tokens (TBT). The prefill phase is highly parallel and compute bound while the decode phase is memory bound. Due to the parallel processing of a large number of tokens, the latency of a prefill iteration is generally higher than that of a decode iteration.

The distinct computational characteristics of prefill and decode operations create a throughput-latency tradeoff in LLM inference [23, 35, 46, 65], as illustrated in Figure 2. Since

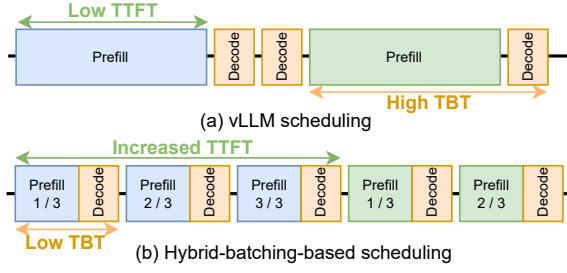


Figure 2. Impact of scheduling strategies on TTFT and TBT.

decoding is memory bound, using a large batch size improves throughput. The original vLLM scheduler [41] uses prefill-prioritizing scheduling to maximize the decode batch size (Figure 2(a)). This approach provides low TTFT, but at the cost of high TBT because a new request’s prefill can pause ongoing decodes, causing *generation stalls* [23]. High TBT is especially problematic in long-context scenarios, where each generation stall can last several seconds.

The issue of high TBT has been acknowledged in real-world deployments [14]. Sarathi-Serve [23] proposed *chunked-prefills* coupled with *continuous hybrid batching* [62] — a technique that divides the prefill tokens of a request into multiple smaller chunks and schedules one prefill chunk per-iteration with on-going decodes (Figure 2(b)). This way, Sarathi-Serve enables increasing batch size while avoiding generation stalls, improving both performance and user interactivity. Various LLM serving systems have incorporated this technique [2, 64, 66], including vLLM [18].

In the common case with hybrid batching, an executing batch consists of one prefill chunk of a pre-determined size and multiple decodes (as shown in Table 1). For example, consider a workload where each request consists of 2K prefill tokens and generates 200 output (decode) tokens. If the prefill chunk size is 1K, a request’s prefill completes over two iterations (prefill tokens / chunk size). Upon completion of the prefill phase, it must execute for another 200 iterations — each iteration corresponding to one output token. In these 200 iterations, 100 requests can complete their prefill phase to join the running batch. This leads to an effective batch size of 101 in the steady state wherein 100 requests execute in their decode phase alongside one prefill chunk of a new request. Executing these hybrid batches requires both high compute (for the prefill chunk) and high memory bandwidth (for the decode requests).

Figure 3 shows how hybrid batching works in practice. Except attention, all other operations are linear i.e., computed element-wise. Linear operations obey the rule $f(x + y) = f(x) + f(y)$ so inputs for a linear operation can be combined, computed upon by the same model weights to reduce memory accesses, and then separated. In contrast, attention is a sequence-level operator that is computed between three

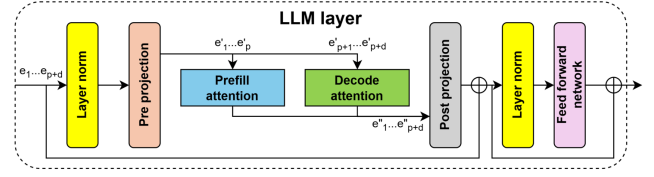


Figure 3. Computation in hybrid batches. Current systems compute prefill inputs ($e_1 \dots e_p$) and decode inputs ($e_{p+1} \dots e_{p+d}$) together for linear operations. However, they compute prefill and decode attention separately using specialized kernels.

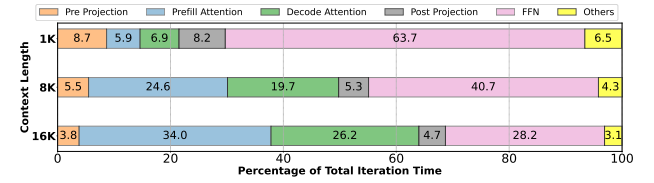


Figure 4. Contribution of different operations in iteration runtime with hybrid batching (model: Llama-3-8B, batch size: 60, chunk size: 1K). For each context length, we show runtime of iteration that processes the last chunk of a prompt.

representations Q (query, of the current tokens being processed), and K/V (key/value, of all tokens in the sequence seen so far) as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\text{scale}}\right)V$$

The QKV representations are further divided among multiple query heads and K/V heads, each assigned to a group [25]. Attention is computed in parallel for each Q head and K/V head pair. Since resource requirements of prefill and decode attention are different, state-of-the-art libraries such as FlashAttention (FA) [29, 30, 49] and FlashInfer (FI) [60] provide specialized kernel APIs, optimized separately for each phase. Use of these kernels works well in small context length scenarios where attention computation is a small fraction of the total inference time [24, 62].

However, the context length in many real-world LLM applications continues to grow [21, 57]. In such scenarios, attention computation dominates, becoming more than 60% of the total inference time in many cases as shown in Figure 4 (context length 16K). Note that prefill and decode attention are computed immediately one after the other in hybrid batches (see Figure 3). Therefore, *when independently optimized attention kernels are used, GPU execution goes through periods of high demand of a resource followed by low utilization of the same resource*. For example, the prefill kernel requires high compute but compute is (mostly) idle when the decode kernel executes.

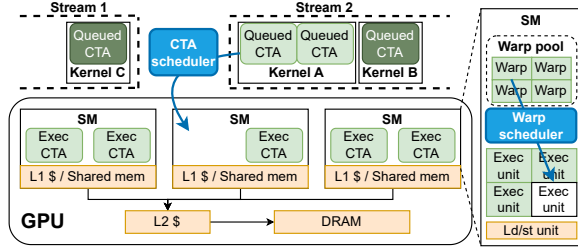


Figure 5. GPU execution model.

We posit that concurrently computing prefill and decode attention can improve performance as it would utilize both compute and memory simultaneously. However, current techniques have several limitations with attention computation. To delve deeper into this, we first explain how GPUs operate and then present a case study of existing methods for executing different operations concurrently on GPUs (§3).

2.2 GPU Execution Model

The GPU’s hardware is arranged in a hierarchy that supports execution at a scale of hundreds of thousands of parallel threads, depicted in Figure 5 [5]. The main processor unit of a GPU is a *Streaming Multiprocessor (SM)*, with modern GPUs containing around a hundred SMs. Each SM has an L1 cache and *shared memory* along with tensor cores for accelerated general matrix multiplication (GEMM) and execution units for integer/floating point operations. The shared memory is a user-addressable partition of the L1 cache. The GPU memory is accessed by SMs through the shared L2 cache.

GPU programming languages expose a hierarchy of threads that mimic the hardware hierarchy. The smallest unit of execution is a thread, while a group of 32 threads make up a *warp*, which typically execute concurrently in lockstep. To maximize throughput, GPU programmers ensure that threads within a warp execute the same code path. A *Co-operative Thread Array (CTA)* [12] is a group of warps that share the L1 cache and shared memory. All warps in a CTA are guaranteed to execute within a single SM.

Users launch GPU *kernels*, or GPU-executed functions, specifying the number of threads in the CTA, the number of CTAs in the kernel, as well as the required shared memory per CTA. This launch is then queued in a *stream*; operations within a stream are serialized but different streams can execute in parallel in any order. The *CTA scheduler* selects CTAs from streams and assigns them to SMs when sufficient execution resources (e.g., threads, shared memory and registers) are available within the SM.

Central to the GPU’s massive throughput is the fast, cycle-level *warp scheduler* baked into the hardware. Every clock cycle, the warp scheduler dispatches eligible warps for execution; a warp is eligible if its threads aren’t stalled (e.g., waiting for memory access). This allows each SM to context

Execution method	GC	WQ	Notes
Streams [45]	×	✓	Easiest to implement
CTA	×	✓	Easy load balancing
Warp (e.g., HFuse [42])	✓	×	Suffers from straggler problem
Intra-thread [53, 59]	✓	×	Cannot overlap with CTA barriers
SM-aware CTA (Ours)	✓	✓	Minimizes operation interference

Table 2. Methods of concurrently executing or fusing different operations along different levels of the GPU execution hierarchy (GC=guarantees op co-location, WQ=reduces wave quantization).

Config.	Description
FA_Serial	Serial execution with FA kernels
FA_Streams	Parallel execution via streams with FA kernels
FA_HFuse	Horizontally fused FA kernels with HFuse [42]
POD (Ours)	Optimized fused computation with our kernel

Table 3. Different methods of computing attention in hybrid batches (FA: FlashAttention).

switch at every clock cycle if required, effectively utilizing all its execution resources.

3 A Case Study on Concurrent Execution

The simplest way to compute prefill and decode attention together is to pass both inputs to an existing attention kernel. Some LLM serving systems prefer this method for computing attention in hybrid batches [7, 17]. In §5.1, we show that this is counter-productive and slower than serial execution.

In this section, we focus on GPU methods for concurrent execution e.g., running kernels in parallel or fusing their operations into a single kernel. We quantitatively analyze their performance and highlight key limitations that motivated us to develop a specialized attention kernel.

3.1 Methods of Concurrent Execution

Each level of the execution hierarchy in a GPU offers potential for concurrent execution (see Table 2).

1. **Kernel-parallel.** Streams can potentially execute different GPU kernels concurrently. This approach is easy to implement as it only requires submitting existing kernels to different streams; all other approaches require fusing different operations into a single kernel. Unfortunately, streams alone guarantees neither concurrency nor SM-level co-location of different operations [45, 63].
2. **CTA-parallel.** In this scheme, the CTAs in the kernel are split across operations in a predetermined manner. CTA-parallel enables better load-balancing: when one CTA finishes execution, the GPU scheduler can deploy the next CTA to the SM. However, similar to streams, CTA-parallel does not guarantee SM-level co-location.
3. **Warp-parallel.** Here, warps within each CTA are split across operations, as proposed in horizontal fusion (HFuse

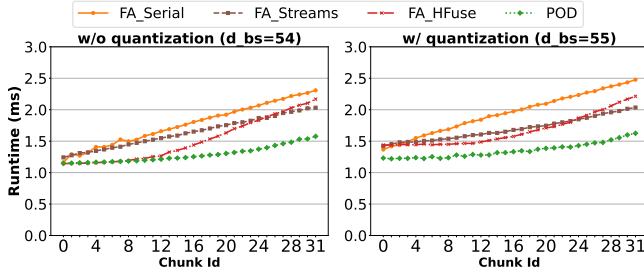


Figure 6. Per layer attention runtime of 32 hybrid batches corresponding to chunked prefills of a request of 16K tokens (chunk size: 512, model: Yi-6B, d_bs : decode batch size).

[42]). This approach guarantees co-location since all warps in a CTA are guaranteed to reside within the same SM. Unfortunately, warp-parallel fusion suffers from the straggler problem: an entire CTA must complete execution before it can be replaced by another one; if one or more of its threads or warps lag behind others, the next CTA is delayed. While fusing the prefill and decode attention computation, the fused kernel requires extensive tuning to deal with a large input space of varying batch sizes and context lengths e.g., some hybrid batches may be prefill heavy and others may be decode heavy. Therefore, a fused prefill-decode attention kernel is particularly vulnerable to the straggler effect with warp-parallel fusion.

4. **Intra-thread.** In intra-thread fusion, each thread alternates between executing instructions of different operations [53, 59]. In simple cases, this strategy provides the maximum opportunity to overlap different operations. However, attention kernels use CTA-level sync barriers to coordinate fetching data into shared memory. These barriers limit intra-thread fusion as instructions before a barrier cannot be overlapped with those after the barrier. We now quantitatively analyze the performance of different methods. Unfortunately, no readily available implementation exists for CTA-parallel and intra-thread fusion. Hence, we first analyze kernel-parallel and warp-parallel methods on attention kernels and then investigate other methods.

3.2 Analysis of Readily Available Methods

For kernel-parallel execution, shown as FA_Streams in Figure 6, we run FA’s prefill and decode kernel on two different CUDA streams. For warp-parallel execution (FA_HFuse), we fuse FA’s kernels using the toolchain provided by [42]. Figure 6 compares their performance against serial execution of FA’s prefill and decode attention kernels (FA_Serial). Our experiment shows the per-layer attention computation time of Yi-6B for 32 chunks of a 16K prompt (chunk size 512), each co-scheduled with decodes of 16K context length each.

Note that if the number of CTAs in a kernel is not divisible by the number of GPU SMs, some of the SMs in the last wave of scheduling can remain idle — a phenomenon known as

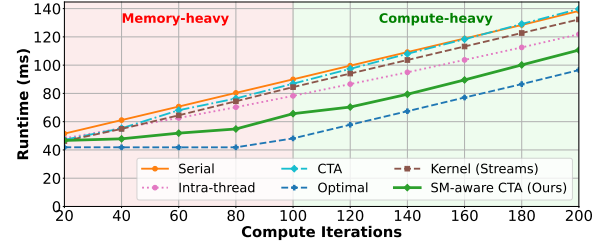


Figure 7. Fine-grained fusion versus serial computation.

wave quantization [38, 44]. In the worst case, a marginal increase in work can double the latency of a kernel due to wave quantization. Therefore, to fully understand the benefit of concurrent execution, we evaluate performance with and without wave quantization. Each decode request uses 4 CTAs in our experiment (one CTA per KV head). Hence a decode batch size of 54 uses 216 CTAs having no wave quantization on our NVIDIA A100 GPU (108 SMs). In contrast, a batch size of 55 uses 220 CTAs leaving 4 quantized CTAs.

FA_Streams provides some speed up over FA_Serial and its gains are higher (up to 20%) when serial execution suffers from wave quantization. This is because streams run kernels in parallel to fill GPU SMs that would otherwise remain idle. This effect can be seen in Figure 6 where FA_Streams take roughly the same amount of time for both batch sizes while the time taken by FA_Serial increases at batch size 55; in particular, decode time increases by more than 25% in FA_Serial when batch size goes from 54 to 55 which increases the total attention time of prefill and decode by up to 17%. FA_HFuse outperforms FA_Streams in some cases but its performance degrades quickly due to straggler effect in the later chunks that are dominated by prefill. This happens because the prefill cost increases with each successive chunk but decode cost is same in all hybrid batches. Overall, FA_Streams and FA_HFuse both perform better than FA_Serial but still leave significant performance on the table as shown by POD-ATTENTION which outperforms both methods by a significant margin.

3.3 Analysis of Other Methods

For complex kernels, such as attention, efficiently implementing fine-grained fusion schemes is non-trivial and prone to errors. Therefore, we analyze the performance of other fusion methods with a simple micro-benchmark consisting of a compute-bound kernel that repeatedly multiplies array elements with a scalar, and a memory-bound kernel that repeatedly adds three arrays. Each thread executes a barrier after each operation. We vary the number of compute iterations to evaluate performance under varying compositions of compute-bound and memory-bound operations. Figure 7 shows the runtime of different fusion methods applied on these two functions. At 100 compute iterations, both operations consume equal time when executed serially. To the

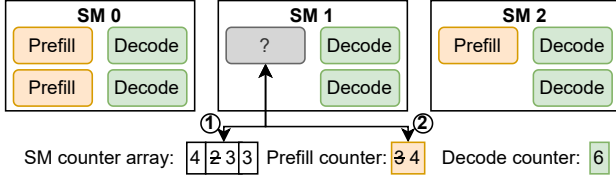


Figure 8. SM-aware CTA scheduling.

left of this point, memory bound is more dominant. To the right, it is compute bound. Figure 7 also shows the runtime achievable with an ideal oracle (i.e., perfect overlap).

CTA and kernel-parallel cannot guarantee SM-level co-location of compute-bound and memory-bound operations and hence provides only marginal average improvement of 3% and 7% over serial execution. Intra-thread fusion outperforms both serial and CTA-parallel execution, on average by 13%. However, the benefit of intra-thread fusion is limited due to sync barriers that hinder concurrent execution.

In summary, current methods for concurrently executing heterogeneous operations face several challenges, such as stragglers, barrier-induced delays, and the inability to guarantee SM-level co-location. In the following sections, we demonstrate how a specialized fused kernel, designed to leverage the characteristics of prefill and decode phases, can overcome these challenges.

4 POD-ATTENTION

We introduce POD-ATTENTION — a single GPU kernel that efficiently computes both prefill and decode attention. Our primary goal is to ensure that each GPU SM computes both operations simultaneously while minimizing resource contention between them. We build our kernel atop FA v2.6.1 [29].

To achieve our goal, we fuse computation along the CTA dimension that helps avoid the pitfalls of finer-grained warp-parallel and intra-thread fusion. In particular, CTA-parallel fusion offers three advantages: 1) it allows different CTAs to start and finish at different times independently of others, 2) ensures that sync barriers do not affect other parts of the computation since the effect of a barrier is limited to within its CTA, and 3) it is easier to program (§4.3). However, naive CTA-parallel fusion cannot guarantee that prefill and decode will be co-located on GPU SMs. To overcome this limitation, we introduce *software-based SM-aware CTA scheduling wherein each CTA decides whether to compute prefill or decode after it has been dispatched to an SM*.

4.1 SM-aware CTA Scheduling

SM-aware CTA scheduling co-locates prefill and decode CTAs through “runtime operation binding”. Here, a CTA decides whether to perform prefill or decode at runtime, after checking: 1) which SM it got launched on [56], and 2) what other CTAs running on the same SM are doing. This

```

1  if (threadIdx.x == 0) { // Leader thread finds assignment
2      int sm_id; // Find which SM this CTA is on
3      asm volatile("mov.u32 %0, %%smid;" : "=r"(sm_id));
4      // For this SM, what do we want to run?
5      const int ratio = (prefill_ratio + decode_ratio);
6      int op, ticket = (atomicAdd(&sm_ctr[sm_id], 1) % ratio);
7      if (ticket < prefill_ratio) op = PREFILL;
8      else op = DECODE;
9      // Get the next CTA for operation
10     int cta_id = atomicAdd(&cta_assign[op], 1);
11     // If the CTA exceeds the max CTA for that op switch ops
12     if (op == PREFILL && cta_id >= prefill_ctas) {
13         op = DECODE;
14         cta_id = atomicAdd(&cta_assign[op], 1);
15     } else if (op == DECODE && cta_id >= decode_ctas) {
16         op = PREFILL;
17         cta_id = atomicAdd(&cta_assign[op], 1);
18     }
19     // Write the CTA ID and operation to shared memory
20     shared_mem[0] = cta_id;
21     shared_mem[1] = op;
22 }
23 __syncthreads(); // Barrier: waits for scheduling to finish
24 // Fetch the assigned CTA and operation.
25 int cta_id = shared_mem[0];
26 const int op = shared_mem[1];
27 __syncthreads();
28 // Perform the appropriate operation
29 if (op == PREFILL) prefill_op(cta_id);
30 else decode_op(cta_id);

```

Figure 9. CUDA code for SM-aware CTA scheduling.

allows the kernel to remain completely agnostic to how the hardware scheduler assigns SMs to CTAs.

To do this, before launching the kernel, we determine how many CTAs are required for prefill and decode independently, and launch the kernel with CTAs matching the sum of both. Each SM has a counter keeping track of the number of CTAs launched on it along with 2 more counters that track the number of prefill and decode CTAs executed on it so far.

Figure 9 shows a simple code snippet of SM-aware CTA scheduling. When the hardware scheduler schedules a new CTA on an SM, a leader thread of the CTA (e.g., thread 0) reads the SMID hardware counter [13] that contains the unique ID of the SM it was launched on (lines 2 - 3). The thread then performs an atomic add operation on the SM counter to obtain a ticket (line 6). This ticket informs the thread as to which operation it should perform i.e., prefill or decode (lines 7 - 8), depending on the scheduling policy. The thread also increments the CTA counter for the operation (line 10). If this exceeds the maximum CTAs for that operation, it switches operations (line 12 - 18). Finally, it writes this information to shared memory so that the other threads in the CTA can begin execution accordingly (lines 20 - 30). We examined two scheduling policies: 50:50 and proportional. In the 50:50 policy, subsequent CTAs on an SM alternate between prefill and decode. In contrast, the proportional policy (line 5) allocates CTAs based on the ratio of prefill and decode CTAs in the current batch.

4.2 Performance Optimizations

Simply co-locating prefill and decode operations does not yield optimal performance. In this subsection, we introduce

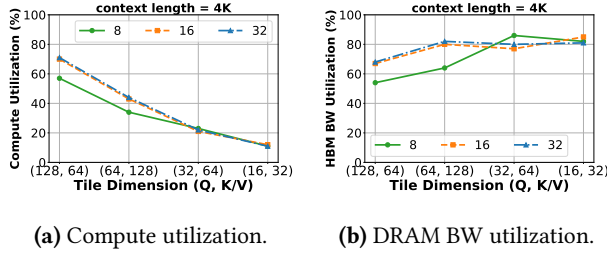


Figure 10. Impact of decode tile size on compute and HBM BW utilization for batch sizes 8, 16 and 32.

various optimizations to maximize the benefit of fusing prefill and decode attention computation.

4.2.1 Tile Sizes. Data tiling is necessary to make effective use of tensor cores, which provide $\sim 8\times$ higher throughput than their CUDA core counterpart [26]. Tiling also helps improve shared memory usage. However, the benefit of tiling is not uniform across operations. Decode operates on a single token per request, having a tile length of one across the query sequence length (QSL) dimension. In Group Query Attention [25], this length increases to the ratio between query and KV heads, typically 2 – 8. Due to this small dimension length, data reuse is insignificant, and performance is limited by memory bandwidth.

FlashAttention uses tile lengths of 64 – 128 for the QSL dimension. The side-effect of using such large tile sizes is that decodes end up zero padded, causing redundant compute [34]. For example, Figure 10a shows that compute utilization of the decode attention kernel is proportional to tile sizes, reaching up to 70% at QSL tile dimension of 128, compared to 10% with tile dimension of 16. However, note that decode attention is memory bound and hence, the primary objective of a decode kernel is to try and saturate memory bandwidth. Figure 10b shows that even at a relatively large QSL tile dimension of 64, the decode kernel is able to maximize memory bandwidth utilization. Hence, for a decode-only attention kernel, there is little incentive to reducing tile sizes further.

In contrast, using large tile sizes for decodes is counter-productive in a fused kernel: any redundant compute performed by decodes interferes with co-located prefills since tensor cores are shared between them. If we reduce unnecessary computation, prefill can make better use of the tensor cores. To do so, we use a decode tile length of 16 for QSL, the minimum needed by CUTLASS [11] for A100 tensor operations. This drops the compute utilization of decodes to $\sim 10\%$, freeing up tensor cores for prefill. Figure 10b shows that reducing tile size has no adverse impact on decode performance at large batch sizes.

4.2.2 Concurrent CTAs per SM. The number of CTAs running concurrently on an SM dictates the amount of resources (e.g., shared memory) each CTA can have. More CTAs per SM implies less resources per CTA, but more opportunities for fine-grained scheduling and co-location, i.e., with 2 CTAs per SM we can only co-locate prefills and decodes in a 1:1 ratio, but with 4 CTAs per SM, we can allocate CTAs to prefill and decode in different proportion depending on batch composition e.g., 3 CTAs to prefill and 1 CTA to decode. In general, prefills benefit from fewer CTAs per SM as it allows each CTA access to more shared memory, enabling use of larger tile sizes. In contrast, decodes do not benefit from larger tile sizes and therefore using more CTAs per SM can be beneficial since it allows fine-grained scheduling.

To achieve the best of both worlds, POD-ATTENTION supports two configurations: 2 CTAs per SM for prefill-dominant hybrid batches and 4 CTAs per SM otherwise. Based on the desired configuration, we modify the tile lengths and number of threads used for prefill and decode. We also explored if 8 CTAs per SM can further improve performance and found that it only marginally improves performance in a few cases while under-performing in most cases. POD-ATTENTION automatically picks the most suitable configuration at runtime.

4.2.3 Virtual Decode CTAs. The amount of shared memory provided to each prefill and decode CTA must be same in the fused kernel. However, because decode uses smaller tile sizes, the shared memory requirement of decode is a quarter of the prefill requirement. To avoid over-allocating shared memory to decodes, we divide each decode CTA into virtual CTAs containing a warp of threads. If the original decode CTA has four warps, each virtual CTA contains one warp which uses a quarter of the shared memory of the original CTA. The sum of shared memory used by all the virtual CTAs in each regular CTA is close to the shared memory used by prefill. This way, virtual decode CTAs balance the shared memory used by prefill and decode.

4.2.4 Limiting Prefill Splits. FlashAttention parallelizes computation across the query heads and QSL tile dimension. FlashDecoding [31], designed for decode which has a QSL of one, further splits the computation across the K/V dimension when there is not enough parallelism to fill the SMs of the GPU. The side-effect of this approach is that different CTAs fetch the same query tensor from memory independently of each other, proportional to the number of splits. Consequently, splitting the computation increases memory bandwidth utilization. While splitting along the key/value dimension is not required for prefills when the input contains enough tokens, chunked-prefills limit the number of tokens processed per-iteration *by design* (to minimize TBT). Therefore, FlashAttention also uses the FlashDecoding technique to accelerate the chunked-prefill attention computation. This scheme works well for a prefill-only kernel as increased parallelism can easily offset the cost of extra memory reads.

However, in a fused kernel, using a large number of splits for chunked-prefills can cause memory bandwidth contention between prefill and decode CTAs, potentially negating the benefit of fusion. To balance this trade-off, we limit the number of splits for a chunked-prefill to fill at most two full waves (determined empirically). This allows a chunked-prefill to use more CTAs when required, while ensuring that the number of splits do not get excessive and harm concurrent decodes.

4.3 Implementing CTA-parallel Fusion

To fuse the two kernels, we first convert them into generic device functions callable from within GPU code while removing all references to the CUDA-provided CTA ID (i.e., *blockIdx*), instead passing this as a function parameter. We build a wrapper kernel that calls these different functions using a calculated CTA ID. The prefill and decode operations execute as if the supplied CTA ID was their actual ID. This enables flexible remapping of CTA IDs, e.g., CTA 0 of the fused kernel can invoke prefill with CTA ID 0, CTA 1 can call decode with ID 0, CTA 2 can call prefill with ID 1, and so on. The amount of shared memory each CTA gets is fixed at kernel launch time, and prefill and decode operations have different requirements. To manage this, we hand-tune the shared memory usage of both prefill and decode operations to balance their requirements while minimizing performance degradation. We launch our fused kernel with enough shared memory for the maximum needed by either operation. To implement virtual CTAs, we modify the decode function replacing all CTA-level barriers with warp-level barriers. The decode function in the fused kernel is called with the appropriate virtual CTA ID, instead of the assigned CTA ID.

4.4 Discussion on Alternative Implementations

Concurrent execution is a well studied topic in GPU literature [40, 43, 55, 61], and our high-level goal of overlapping prefill and decode attention computation can be achieved in multiple ways. One noteworthy strategy is based on persistent threads [32, 45, 63]: in this method, one launches a pre-determined number of CTAs (enough to perfectly fill all the SMs). Persistent threads of these CTAs pull the right type of work as necessary (e.g., prefill or decode tiles). We find that this strategy also alleviates the straggler problem. However, SM-aware scheduling is still needed to decide what work (prefill or decode) to run on which persistent CTA, critical to guaranteeing operation co-location within an SM. Upon integrating it with SM-aware scheduling, we find that this strategy performs on par with our CTA-parallel fusion.

NVIDIA also provides MPS (multi-process service) [10] and MIG (multi-instance GPUs) [9] features to run different applications in parallel on the same GPU. However, because hybrid batching combines prefill and decode operations within a single process by design, MPS and MIG are inapplicable to our use case.

Model	GPU	# Q Heads	# KV Heads	# Layers
Yi-6B	1 A100	32	4	32
Llama-2-7B	2 A100s	32	32	32
Llama-3-8B	2 A100s	32	8	32

Table 4. Models and hardware used for evaluation.

5 Evaluation

Our evaluation answers the following questions:

- What is the effect of POD-ATTENTION on attention computation latencies?
- How does POD-ATTENTION affect end-to-end LLM inference performance?
- What is the impact of different optimizations and design choices employed in POD-ATTENTION?

Models and environment: We evaluate POD-ATTENTION with Yi-6B (4 KV heads [20]), Llama-2-7B (32 KV heads [6]) and Llama-3-8B (8 KV heads [8]), deploying Yi-6B on one A100 GPU, and others on two A100 GPUs with tensor parallelism (Table 4). Each model has 32 query heads. Each GPU has 80GB HBM memory.

Workloads and metrics: We evaluate both offline and online inference scenarios. For offline inference, we report the number of requests processed per minute. For online inference, we report TTFT, TBT and request execution latency on two workloads consisting of 2K requests each, and context length ranging from 4K to 32K tokens per-request. One of the workloads is an internal enterprise workload (mean context length of 10.5K tokens, per-request prefill to decode token ratio i.e., P:D in the range of 0 – 40) and the other is based on arXiv-Summarization [4] (mean context length of 9.5K tokens, P:D ratio of 0-50). On average, the number of decode tokens in arXiv workload is 42% higher (470) than the internal workload (331).

Serving system baselines: Our experiments use Sarathi-Serve [15] as the serving framework, which is built atop vLLM [19]. We evaluate two baselines: 1) the original vLLM scheduler [41] that runs prefills and decodes in separate batches, prioritizing prefills over decodes and 2) Sarathi-Serve [23]. Both baselines use FlashAttention kernels (v2.6.1) for attention computation. We integrate POD-ATTENTION into Sarathi-Serve to evaluate the benefits of our optimizations. For simplicity, we refer to Sarathi-Serve without and with POD-ATTENTION as Sarathi and Sarathi+POD.

5.1 Evaluating Attention Computation

Figure 6 illustrates a specific instance where POD-ATTENTION accelerates attention computation, outperforming the next best alternative by up to 29%. To demonstrate the broad applicability of POD-ATTENTION, we conducted a comprehensive sweep across over a thousand hybrid batches on our models. In these experiments, we varied the context length from 4K to 20K and the prefill chunk size from 512 to 2K. We focused

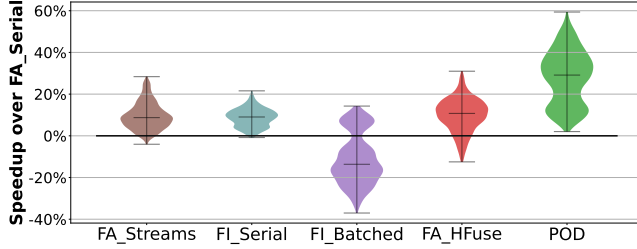


Figure 11. Distribution of speedup in attention computation with different mechanisms compared to FA_Serial.

on scenarios where prefill and decode attention account for at least 20% of the serial runtime, as other cases offer limited potential for optimization through operation fusion.

In addition to FlashAttention kernels, we also compare the runtime of FlashInfer (FI) v0.2.0 kernels [60] in two configurations: FI_Serial and FI_Batched. FI_Batched computes prefill and decode attention using the prefill kernel of FlashInfer. We compare against FI_Batched for two reasons: 1) this strategy is the easiest way to compute prefill and decode attention together, and 2) some systems prefer this method e.g., Sarathi used FI_Batched in its default attention back-end [7], and a similar feature is requested in vLLM [17]. However, we show that this strategy is inefficient e.g., when FI_Batched uses a prefill-optimized kernel, it leads to redundant compute in decode computation due to use of larger tile sizes (§4.2.1). This redundant computation interferes with co-running prefill. Similar interference occurs on memory-bandwidth if FI_Batched uses a decode-optimized kernel.

Figure 11 shows the relative speedup for different mechanisms compared to FA_Serial. FA_Streams provides limited speedup as it cannot guarantee SM-level overlap of operations. In rare cases, we find that the overhead of stream synchronization can also negate its benefits. FI_Serial has better optimized decode kernels giving it a modest improvement over FA_Serial, but it does not overlap the operations. FI_Batched improves performance at low context lengths, but degrades at higher lengths by up to 40% due to redundant computation for decodes. FA_HFuse is the strongest baseline as it guarantees operation overlap, improving median performance by 11%. However, FA_HFuse is susceptible to the straggler effect due to which it is slower by up to 13% compared to FA_Serial. The straggler effect can also be seen in Figure 6 towards the later chunks where prefill is more dominant, making it hard to achieve perfect utilization.

POD-ATTENTION reaches a peak speedup of 59%, and a mean of 28% — higher than all alternatives. We found that in 25% of cases, it also reaches within 10% of the theoretical peak speedup, signifying near-perfect overlap. Furthermore, unlike other alternatives, POD-ATTENTION never under-performs serial execution. These results underline the importance of a specialized attention kernel for hybrid-batching-based LLM inference.

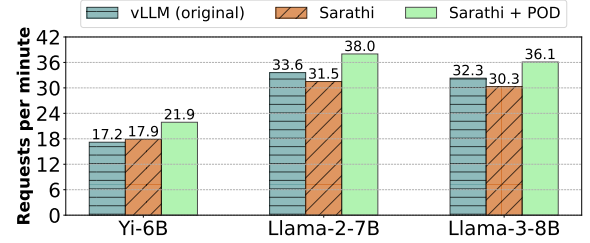


Figure 12. Serving throughput in offline inference.

Additionally, we profiled the energy consumption of the attention kernels and observed that POD-ATTENTION reduces energy consumption by up to 35% over FA_Serial (mean 20.5%). These savings are largely proportional to the reduction in runtime, showing that prefill-decode overlap not only improves performance but also reduces energy consumption.

5.2 Evaluating Throughput in Offline Inference

For evaluating offline inference scenarios, we run long context requests of 16K tokens each. We use chunk size 512 for Yi-6B, and 1K for both Llama-2-7B and Llama-3-8B, chosen in a way that chunking a prompt does not reduce the performance of linear operations (as recommended by Sarathi [23, 24]). We run 1K total requests for Yi-6B, and 2K requests each for Llama-2-7B and Llama-3-8B such that the total runtime of a single configuration is about one hour. The number of output tokens per-request is set to 2K for Yi-6B, 1K for Llama-3-8B and 256 for Llama-2-7B; we study the effect of varying prefill to decode token ratio (P:D ratio) in §5.4.4.

Figure 12 shows that Sarathi+POD delivers the best throughput: 22%, 20% and 19% higher than Sarathi, and 27%, 13% and 12% higher than vLLM, for the three models. It is worth highlighting that chunked-prefills and hybrid batching involves a tradeoff. Chunking a prompt increases attention computation time due to repeated KV cache loads: computing attention of a prefill chunk requires reading KV cache of all prior chunks [65]. At the same time, fusing decode tokens with prefills helps execute linear operations more efficiently: model weights need not be read separately for prefills and decodes. Therefore, the relative performance of vLLM and Sarathi can vary depending on workload, model configuration and chunk size. In our experiments, Sarathi improves throughput slightly over vLLM for Yi-6B but underperforms it for Llama-2-7B and Llama-3-8B. Sarathi+POD fuses prefills and decodes in all operations to improve GPU resource utilization, thereby outperforms both baselines.

5.3 Evaluating Latency in Online Inference

We evaluate Llama-3-8B on the internal and arXiv-based workloads near the serving capacity of the system: the maximum load a system can handle while avoiding high queuing delays [23]. We evaluate 2048 requests in each workload by

QPS	System	TTFT		TBT		Request Latency		% Requests with Stalls	
		P50	P99	P50	P99	P50	P99	200ms	500ms
1.1	vLLM (original)	0.67	10.11	0.04	1.13	25.05	91.01	99.95	97.8
	Sarathi	2.2	12.58	0.10	0.15	26.83	92.24	2.05	0
	Sarathi+POD	1.9	12.26	0.10	0.14	24.70	79.04	3.17	0
1.2	vLLM (original)	0.94	12.70	0.07	1.76	42.73	151.8	99.95	99.6
	Sarathi	25.44	57.83	0.12	0.16	67.12	140.5	5.07	2.63
	Sarathi+POD	7.49	23.78	0.11	0.15	38.69	106.8	2.29	0

Table 5. Internal workload. Latency numbers in seconds.

QPS	System	TTFT		TBT		Request Latency		% Requests with Stalls	
		P50	P99	P50	P99	P50	P99	200ms	500ms
0.85	vLLM (original)	0.55	6.26	0.03	0.82	20.53	234.93	99.9	97.8
	Sarathi	2.68	14.89	0.08	0.13	27.87	281.07	4.15	2.05
	Sarathi+POD	1.85	12.71	0.08	0.11	24.31	255.75	1.85	1.61
0.95	vLLM	0.71	8.25	0.06	1.36	36.86	401.2	99.9	99.45
	Sarathi	46.22	144.2	0.1	0.14	90.12	417.6	4.44	1.9
	Sarathi+POD	11.74	27.38	0.09	0.12	40.6	333.0	2.2	2.1

Table 6. arXiv-based workload. Latency numbers in seconds.

varying the input load based on Poisson distribution. For Sarathi and Sarathi+POD, we use chunk size of 1024 for the arXiv-based workload, and 1536 for the internal workload which is more prefill-heavy. We discuss performance on important LLM-specific latency metrics of TTFT, TBT, and end-to-end request execution latency.

Note that there is an inherent trade-off between these metrics [23] and optimizing for one metric can severely compromise the others. For example, as will see below, vLLM prioritizes prefills and thus achieves low TTFT but sacrifices TBT, resulting in 95+% of user requests experiencing one or more stalls during decode generation. On the other hand, Sarathi reduces the stalls to a small % of user requests but significantly increases TTFT compared to vLLM.

5.3.1 TTFT. vLLM provides the lowest TTFT as it schedules a prefill on the first available opportunity. In comparison, Sarathi increases TTFT because the ongoing decodes interfere with prefills. TTFT in Sarathi further increases with the load, particularly due to higher queuing delays, e.g., the median TTFT goes to 25.4 and 46.2 seconds for the internal and arXiv-based workloads, compared to 0.94 and 0.71 seconds of vLLM. Sarathi+POD significantly reduces TTFT over Sarathi, bringing the median TTFT down to 7.5 and 11.74 seconds at higher load. Sarathi+POD also reduces the P99 TTFT by up to 4.3× over Sarathi.

5.3.2 TBT and Stalls. vLLM induces generation stalls by pausing on-going decodes whenever a new prefill is scheduled, resulting in poor interactivity with the LLM service. These generation stalls are reflected as high tail TBT latency, e.g., the P99 TBT of vLLM reaches up to 1.76 seconds (internal workload) and 1.36 seconds (arXiv-based workload).

Latency Metric	vLLM (original)	Sarathi+POD		
		1024	1536	2048
TTFT (P50)	0.67	6.29	1.9	1.59
TTFT (P99)	10.11	18.99	12.26	12.40
TBT (P50)	0.04	0.08	0.10	0.08
TBT (P99)	1.13	0.11	0.14	0.18

Table 7. TTFT and TBT of Sarathi+POD with different chunk sizes versus vLLM (internal workload, QPS 1.1).

In the worst-case, we observe that the highest TBT latency reaches up to 8 seconds in vLLM when it computes multiple prefills consecutively. In comparison, Sarathi ensures that ongoing decodes do not get affected by a new prefill. Therefore, Sarathi provides significantly lower tail TBT latency compared to vLLM e.g., the P99 TBT of Sarathi is at most 0.16 seconds (10× lower than vLLM). Sarathi+POD further minimizes tail TBT over Sarathi by 10 – 20%. Crucially, since a single response results in a large number of decodes, *high TBT tail latency affects nearly all requests in vLLM*, signifying poor interactive experience for almost all users. Even if the TBT SLO is raised to 500ms, more than 97% of the total requests experience at least one stall in vLLM. In contrast, very few requests (<5%) observe a stall in Sarathi, which Sarathi+POD further reduces in most cases.

5.3.3 End-to-end Request Latency. Request latency can be used to approximate system throughput in online inference. Sarathi reduces P99 request latency over vLLM by 8% for the internal workload at QPS 1.2, but increase it by up to 24% over vLLM for the arXiv-based workload (QPS 0.85). Sarathi+POD is not only better than Sarathi in all cases, but

CL	Batch size					CL	Batch size				
	8	16	32	64	128		8	16	32	64	128
1024	1.08	1.00	1.07	1.14	1.03	1024	1.00	1.00	1.00	1.00	1.00
2048	1.00	1.00	1.00	1.09	1.05	2048	1.10	1.06	1.05	1.00	1.00
4096	1.00	1.00	1.00	1.00	1.00	4096	1.13	1.12	1.11	1.05	1.04
8192	1.00	1.00	1.00	1.00	1.00	8192	1.13	1.15	1.16	1.11	1.08
16384	1.00	1.00	1.00	1.00	1.00	16384	1.17	1.16	1.17	1.17	1.15

(a) 2 CTAs per SM.

(b) 4 CTAs per SM.

Figure 13. POD-ATTENTION with varying CTA configs.

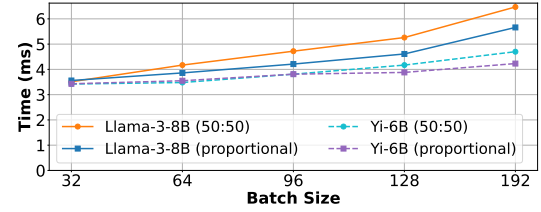
also outperforms vLLM in many cases e.g., it reduces the P99 request execution latency by up to 42% over vLLM for the internal workload (106.8 seconds vs 151.8 seconds at QPS 1.2) and by up to 17% for the arXiv-based workload (333 seconds vs 401.2 seconds at QPS 0.95).

These results demonstrate that Sarathi enhances interactivity by reducing tail TBT and minimizing stalls, albeit with increased TTFT and some throughput reduction compared to vLLM. POD-ATTENTION optimizes Sarathi’s performance across all metrics, effectively balancing the throughput-latency tradeoff. Table 7 shows that the chunk size in Sarathi+POD can be tuned further to navigate the TTFT and TBT trade-off, e.g., using a larger chunk size of 2K tokens lowers the median TTFT from 6.3 seconds to 1.6 seconds at the cost of higher TBT (P99 0.18 seconds vs 0.11 seconds).

5.4 Sensitivity Studies

5.4.1 CTAs per SM. Figure 13 shows the performance of POD-ATTENTION with different numbers of CTAs running concurrently on an SM, varying batch sizes (horizontally) and context lengths (vertically) for Llama-3-8B. For each (context length, batch size) data point, we normalize the runtime to the best among the two configurations. In general, for long contexts where prefill cost dominates, 2 CTAs per SM performs better as it allows for larger tile sizes. As the context length decreases, the decode cost starts dominating and hence 4 CTAs per SM starts performing better: more CTAs per SM allows packing more decodes with fewer prefills, e.g., 1 prefill CTA and 3 decode CTAs.

5.4.2 Scheduling Policy. We explore two CTA scheduling policies within an SM, namely 50:50 allocation and Proportional allocation. In 50:50 allocation, CTAs launched on an SM alternate between prefill and decodes, i.e., the first CTA performs prefill, the next decode, and so on. This policy is agnostic to the total number of prefill and decode CTAs in the kernel. In Proportional allocation, the CTAs pick whether to perform prefill or decode depending on the total number of CTAs in the kernel. For example, if 50 prefill and 100 decode CTAs are required, the first CTA on each SM will perform prefill, the next two CTAs will perform decode, then repeat. Figure 14 shows the latency of POD-ATTENTION with these policies for 8K context length and varying decode batch sizes on Yi-6B and Llama-3-8B. We notice that

**Figure 14.** Effect of scheduling policy in POD-ATTENTION.

Chunk Id	FA_Serial	POD-ATTENTION	
		Vanilla split	Limited split [Ours]
28	1.93	1.68 (0.87×)	1.45 (0.75×)
29	1.96	1.69 (0.86×)	1.45 (0.74×)
30	1.98	1.71 (0.86×)	1.45 (0.73×)
31	1.99	1.71 (0.86×)	1.46 (0.73×)

Table 8. Per-layer attention runtime (ms) of last four prefill chunks of a prompt, co-running with decode batch size 64 (model: Llama-3-8B, context length: 16K, chunk size: 512).

as the load increases (greater batch size), the performance of Proportional improves over 50:50 allocation. Proportional allocation spreads out the less frequent operations allowing better operational overlap and reduced resource contention, performing up to 14% better than a 50:50 allocation scheme.

5.4.3 Limiting Prefill Splits. POD-ATTENTION reduces attention computation time with the default FlashDecoding-style splitting along the KV dimension. However, limiting the number of splits further improves performance. For example, Table 8 shows that in the last four chunks of a 16K prompt, co-running with 64 decode requests of the same context length, limiting the number of splits in prefill attention computation nearly doubles the speedup of POD-ATTENTION over FA_Serial.

5.4.4 Sensitivity to Workload. POD-ATTENTION accelerates the execution of hybrid batches and hence its impact on overall performance depends on how many iterations consist of hybrid batches in a given workload. A workload that is highly dominated by either prefills (high P:D ratio) or decodes (low P:D ratio) is likely to experience little benefit with POD-ATTENTION. To understand the effect of varying P:D ratio, we benchmark Llama-3-8B with a total of 2048 requests, each consisting of ≈ 16.5 K tokens, but with varying P:D ratio (in the range of 8 to 24) e.g., if the P:D is 10, then a request contains ≈ 15 K prefill tokens and ≈ 1.5 K decode tokens. Figure 15 shows that Sarathi+POD outperforms Sarathi over varying workload mixes. The peak gains occur in the P:D range of 12 to 18 because most batches are hybrid batches in this regime. In contrast, many iterations run decode-only batches when P:D ratio is lower than 12 (or prefill-only batches when P:D ratio is higher than 18).

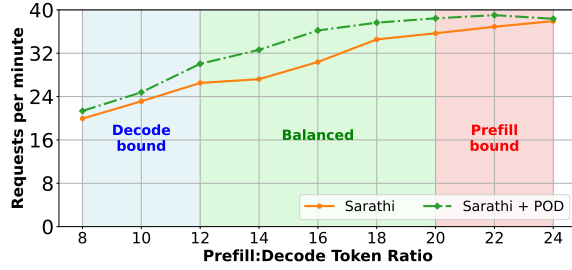


Figure 15. Request processing throughput under varying workload distribution (model: Llama-3-8B, TP-2).

6 Related Work

Optimizing GPU execution and LLM serving systems is an active area of research [21, 23, 27, 33, 35, 36, 38, 39, 41, 46–48, 52, 57, 62, 64–66].

Optimizing Attention Computation: FlashAttention [30] introduced the first specialized implementation of attention, fusing all its operations into a single kernel with tile-based computation. FA-2 [29] improved it further with better work partitioning and load balancing. FlashDecoding [31] accelerates decode attention by splitting computation along the KV dimension. FlashDecoding++ [34] uses asynchronized softmax, double-buffered flat GEMM optimizations, and dataflow-based hardware resource adaptation to accelerate decode. LeanAttention [48] follows Stream-K reduction [44] of tiled calculation to enable better load distribution across SMs for decodes. FlashInfer [60] introduced shared-prefix based optimized attention kernels. Compared to works that separately handle prefill and decode, POD-ATTENTION jointly optimizes and fuses them into a single kernel.

FA-3 [49] is a recent addition to the FlashAttention family of kernels. It leverages new features available in the NVIDIA Hopper architecture, exploiting the asynchrony of Tensor Cores, the Tensor Memory Accelerator, and the Special Function Units. FA-3 was under active development at the time of writing this paper and hence we leave extending POD-ATTENTION support to FA-3 and Hopper architecture for future work.

Operation Fusion: Kernel fusion is a commonly used technique for improving GPU performance. Elastic kernels [45] proposes restricting resources to enable running multiple kernels concurrently. However, this method provides no guarantee of intra-SM co-location. To overcome this, ISPA [63] deploys a predetermined number of CTAs for each kernel, less than the number of CTAs that run concurrently on the GPU. Significant a priori profiling is used to determine the appropriate CTA sizes to allow for both kernels to execute concurrently. This can be tedious for attention kernels with dynamically changing input sizes, and makes load balancing between the prefill and decode operations difficult, as one

operation completing early leaves resources underutilized. HFuse [42] fuses operations in warp-parallel fashion, providing source-to-source compilation tools to fuse kernels. SM-centric scheduling [56] uses the SM counter to assign work to CTAs, which we leverage in POD-ATTENTION.

Optimizing LLM Inference: Optimizing LLM serving systems is an active area of research [23, 33, 36, 41, 46, 50–52, 57, 58, 62]. Orca [62] introduced iteration-level scheduling to eliminate compute fragmentation when requests of different lengths are batched together. PagedAttention [41] and vAttention [47] proposed different techniques for dynamic memory management for LLM inference. Sarathi-Serve [23] leverages chunked prefills to enable stall-free batching. In contrast, Splitwise [46], DistServe [65] and TetriInfer [35] disaggregate the prefill and decode phases onto different GPU nodes to avoid interference between these phases. Various recent works have also proposed overlapping compute with communication to improve resource utilization [28, 37, 54].

Similar to POD-ATTENTION, NanoFlow [66] also targets improving intra-device resource utilization, albeit with a contrasting approach. NanoFlow divides a batch into smaller operation-level nano-batches and schedules them in a way that overlaps operations with complementary resource profiles via CUDA streams. In contrast, POD-ATTENTION tries to maximize resource-utilization within a given batch by fusing prefill and decode attention computation. While NanoFlow requires large batch sizes in order to benefit from batch splitting, POD-ATTENTION is useful when attention consumes a significant amount of time. Therefore, NanoFlow seems more suitable for small-context scenarios whereas POD-ATTENTION targets long-context scenarios that depend on hybrid batching for efficient LLM serving.

7 Conclusion

We introduce POD-ATTENTION — the first attention kernel specialized to compute prefill and decode attention in parallel such that both compute and memory bandwidth of a GPU can be utilized simultaneously. POD-ATTENTION enables efficient hybrid batching based LLM inference by accelerating attention computation by up to 59% (mean 28%) compared to using independently optimized prefill and decode attention kernels. POD-ATTENTION also improves the end-to-end serving throughput by up to 22%, while significantly reducing latency over state-of-the-art LLM serving systems Sarathi-Serve and vLLM.

Acknowledgments

We thank our shepherd Tim Rogers, the anonymous ASPLOS reviewers and Ajay Nayak for their valuable feedback on various aspects of the paper. We also thank Zihao Ye for various helpful discussions on POD-ATTENTION and FlashInfer. Aditya K Kamath and Simon Peter are supported by National Science Foundation grant CNS-2212580.

A Artifact Appendix

A.1 Abstract

POD-ATTENTION is a GPU kernel that overlaps prefill and decode attention operations for large language models. POD-ATTENTION is built on top of FlashAttention kernels (v2.6.1) [29] and is integrated with Sarathi-Serve [23] – a state-of-the-art hybrid batching based LLM inference scheduler.

A.2 Artifact check-list (meta-information)

- **Compilation:** CUDA 12.4, GCC 11.4.
- **Model:** Llama-2-7B [6], Llama-3-8B [8], Yi-6B [20].
- **Data set:** arXiv-Summarization [4].
- **Run-time environment:** Ubuntu 22.04, CUDA 12.4, Python 3.12, and PyTorch 2.4.
- **Hardware:** 1–2 NVIDIA A100 80 GB GPUs, x86 machine.
- **How much time is needed to prepare workflow?:** 1 minute with Docker image. 1–2 hours if installing from source.
- **How much time is needed to complete experiments (approximately)?:** Approx. 18 hours.
- **Publicly available?:** Yes.
- **Archived (provide DOI)?:** 10.5281/zenodo.14770841

A.3 Description

A.3.1 How to access. We provide the source code in various forms: Docker container (see A.3.3), GitHub repository (https://github.com/microsoft/vattention/tree/main/pod_attn), and Zenodo (<https://doi.org/10.5281/zenodo.14770840>).

A.3.2 Hardware dependencies. This artifact requires an x86 machine with 2 NVIDIA A100 GPUs with 80GB memory each. If only one GPU is available, all experiments can be conducted in full, except for Table 6 and the results for Llama-2-7B and Llama-3-8B in Figure 12.

A.3.3 Software dependencies. POD-ATTENTION has been tested on a machine with Ubuntu 22.04. All other software dependencies are resolved while installing.

A.3.4 Data sets. Some experiments are based on the arXiv-Summarization dataset. We use a subset of the dataset available in the traces/ folder of the artifact.

A.3.5 Models. This artifact evaluates Yi-6B, Llama-2-7B and Llama-3-8B. Accessing Yi-6B and Llama-2-7B is straightforward but accessing Llama-3-8B requires logging into huggingface with the user’s private token (HF_TOKEN below):

```
$ huggingface-cli login --token HF_TOKEN
```

A.4 Installation

We provide two methods of installing and testing: using Docker (recommended) or manual installation.

A.4.1 Docker installation (recommended). We provide a docker image for POD-ATTENTION with all its dependencies pre-installed. You can launch the docker container and navigate to the artifact directory as follows:

```
$ docker run --gpus all -it \
  -p 8181:8181 --rm --ipc=host --cap-add=SYS_ADMIN \
  rnp1910/pod_attention:asplos_25_pytorch_run
$ cd /workspace/vattention/pod_attn
```

A.4.2 Manual installation. For manual installation, we can download POD-Attention (available in vAttention repository) to home directory to install it. We use Anaconda for the appropriate versions of CUDA, Python, and PyTorch. This can take up to 2 hours.

```
$ git clone \
  https://github.com/microsoft/vattention.git
$ cd vattention/pod_attn/
# Install miniconda; skip if already installed
$ make install_miniconda
$ bash # Refresh shell and activate
$ conda activate pod_attn
# Install CUDA Toolkit
(pod_attn)$ conda install -y -c \
  conda-forge cuda-toolkit=12.4.0
# Install dependencies
(pod_attn)$ pip install -r requirements.txt
(pod_attn)$ pip install flashinfer==0.1.5 \
  -i https://flashinfer.ai/whl/cu124/torch2.4
# Install POD-Attention and vAttention
(pod_attn)$ make install_all
```

A.5 Experiment workflow

The source code for POD-ATTENTION kernel is available in the vattention/pod_attn/ folder. Our evaluation primarily contains two kinds of experiments: attention performance (Figures 1, 6, 10, 11, 13, 14) and end-to-end LLM performance (Figure 12 and Table 6). Figure 7 evaluates various kernel fusion strategies with a micro-benchmark. Most of these require only one GPU except for Table 6 and Figure 12 (for Llama-2-7B and Llama-3-8B) that require two GPUs. Use the Makefile present in the vattention/pod_attn/ folder to run experiments as follows:

```
make figure1 # 2 minutes; sudo used by script
make figure6 # 2 minutes
make figure7 # 2 minutes
make figure10 # 1 minute; sudo used by script
make figure11 # 2 hours
make figure12 # 9 hours
make figure13 # 1 minute
make figure14 # 1 minute
make table6 # 4 hours
```

A.6 Evaluation and expected results

The artifact scripts redirect the raw output numbers and logs to output/ folder, while the plotted graphs can be found in the graphs/ folder. Tables are saved as CSVs in the same folder. Results may have minor runtime variations from those reported in the paper, but general trends should hold.

References

- [1] 2022. FlashAttention. <https://github.com/Dao-AI/flash-attention>.
- [2] 2023. TensorRT-LLM: A TensorRT Toolbox for Optimized Large Language Model Inference. <https://github.com/NVIDIA/TensorRT-LLM>.
- [3] 2024. AI Infrastructure Spending Forecast to Be Over a Trillion Dollars Over the Next Five Years. <https://www.delloro.com/news/ai-infrastructure-spending-forecast-to-be-over-a-trillion-dollars-over-the-next-five-years/>.
- [4] 2024. ccdv/arxiv-summarization. <https://huggingface.co/datasets/ccdv/arxiv-summarization>.
- [5] 2024. CUDA C Programming Guide – Hardware Implementation. <https://docs.nvidia.com/cuda/cuda-c-programming-guide/#hardware-implementation>.
- [6] 2024. Llama-2-7B. <https://huggingface.co/meta-llama/Llama-2-7b-hf>.
- [7] 2024. Merged PR 1865: Critical bug fixes related to sampling. <https://github.com/microsoft/sarathi-serve/commit/50e59c51b85b1157e001bb8ee7a1b049d551955d#diff-450b0de5cce8a2341140afed859dc5dd3b913fa6e62d27988fccefeacc7b33ec>.
- [8] 2024. Meta-Llama-3-8B. <https://huggingface.co/meta-llama/Meta-Llama-3-8B>.
- [9] 2024. NVIDIA Multi-Instance GPU. <https://www.nvidia.com/en-us/technologies/multi-instance-gpu/>.
- [10] 2024. NVIDIA Multi-Process Service. <https://docs.nvidia.com/deploy/mps/index.html>.
- [11] 2024. NVIDIA/cutlass: CUDA Templates for Linear Algebra Subroutines. <https://github.com/NVIDIA/cutlass>.
- [12] 2024. Parallel Thread Execution ISA Version 8.5 – Cooperative Thread Arrays. <https://docs.nvidia.com/cuda/parallel-thread-execution/#cooperative-thread-arrays>.
- [13] 2024. Parallel Thread Execution ISA Version 8.5 – Special Registers: %smid. <https://docs.nvidia.com/cuda/parallel-thread-execution/index.html#special-registers-smid>.
- [14] 2024. Performance and Tuning. <https://docs.vllm.ai/en/v0.6.0/models/performance.html>.
- [15] 2024. Sarathi-Serve. <https://github.com/microsoft/sarathi-serve>.
- [16] 2024. The State of AI Infrastructure at Scale 2024. <https://ai-infrastructure.org/wp-content/uploads/2024/03/The-State-of-AI-Infrastructure-at-Scale-2024.pdf>.
- [17] 2024. Unify the kernel used in flash attention backend. <https://github.com/vllm-project/vllm/pull/6052>.
- [18] 2024. Upstream Chunked Prefill. <https://github.com/vllm-project/vllm/issues/3130>.
- [19] 2024. vLLM: Easy, fast, and cheap LLM serving for everyone. <https://github.com/vllm-project/vllm>.
- [20] 2024. Yi-6B-200K. <https://huggingface.co/01-ai/Yi-6B-200K>.
- [21] Amey Agrawal, Junda Chen, Íñigo Goiri, Ramachandran Ramjee, Chaojie Zhang, Alexey Tumanov, and Esha Choukse. 2024. Mnemosyne: Parallelization Strategies for Efficiently Serving Multi-Million Context Length LLM Inference Requests Without Approximations. arXiv:2409.17264 [cs.LG] <https://arxiv.org/abs/2409.17264>
- [22] Amey Agrawal, Nitin Kedia, Jayashree Mohan, Ashish Panwar, Nipun Kwatra, Bhargav S Gulavani, Ramachandran Ramjee, and Alexey Tumanov. 2024. Vidur: A Large-Scale Simulation Framework For LLM Inference. *Proceedings of The Seventh Annual Conference on Machine Learning and Systems, 2024, Santa Clara* (2024).
- [23] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. USENIX Association, Santa Clara, CA, 117–134. <https://www.usenix.org/conference/osdi24/presentation/agrawal>
- [24] Amey Agrawal, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, and Ramachandran Ramjee. 2023. SARATHI: Efficient LLM Inference by Piggybacking Decodes with Chunked Prefills. arXiv:2308.16369 [cs.LG] <https://arxiv.org/abs/2308.16369>
- [25] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebron, and Sumit Sanghai. 2023. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Houda Bouamor, Juan Pino, and Kalika Bali (Eds.). Association for Computational Linguistics, Singapore, 4895–4901. <https://doi.org/10.18653/v1/2023.emnlp-main.298>
- [26] Jeremy Appleyard and Scott Yokim. 2017. Programming Tensor Cores in CUDA 9. <https://developer.nvidia.com/blog/programming-tensor-cores-cuda-9/>.
- [27] Shiyi Cao, Shu Liu, Tyler Griggs, Peter Schafhalter, Xiaoxuan Liu, Ying Sheng, Joseph E. Gonzalez, Matei Zaharia, and Ion Stoica. 2024. MoE-Lightning: High-Throughput MoE Inference on Memory-constrained GPUs. arXiv:2411.11217 [cs.DC] <https://arxiv.org/abs/2411.11217>
- [28] Li-Wen Chang, Wenlei Bao, Qi Hou, Chengquan Jiang, Ningxin Zheng, Yinmin Zhong, Xuanrun Zhang, Zuquan Song, Ziheng Jiang, Haibin Lin, Xin Jin, and Xin Liu. 2024. FLUX: Fast Software-based Communication Overlap On GPUs Through Kernel Fusion. arXiv:2406.06858 [cs.LG] <https://arxiv.org/abs/2406.06858>
- [29] Tri Dao. 2024. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. In *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=mZn2Xyh9Ec>
- [30] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FLASHATTENTION: fast and memory-efficient exact attention with IO-awareness. In *Proceedings of the 36th International Conference on Neural Information Processing Systems* (New Orleans, LA, USA) (NIPS '22). Curran Associates Inc., Red Hook, NY, USA, Article 1189, 16 pages.
- [31] Tri Dao, Daniel Haziza, Francisco Massa, and Grigory Sizov. 2023. Flash-Decoding for long-context inference. <https://crfm.stanford.edu/2023/10/12/flashdecoding.html>.
- [32] Kshitij Gupta, Jeff A. Stuart, and John D. Owens. 2012. A study of Persistent Threads style GPU programming for GPGPU workloads. In *2012 Innovative Parallel Computing (InPar)*. 1–14. <https://doi.org/10.1109/InPar.2012.6339596>
- [33] Connor Holmes, Masahiro Tanaka, Michael Wyatt, Ammar Ahmad Awan, Jeff Rasley, Samyam Rajbhandari, Reza Yazdani Aminabadi, Heyang Qin, Arash Bakhtiari, Lev Kurilenko, and Yuxiong He. 2024. DeepSpeed-FastGen: High-throughput Text Generation for LLMs via MII and DeepSpeed-Inference. arXiv:2401.08671 [cs.PF] <https://arxiv.org/abs/2401.08671>
- [34] Ke Hong, Guohao Dai, Jiaming Xu, Qiuli Mao, Xiuhong Li, Jun Liu, kangdi chen, Yuhang Dong, and Yu Wang. 2024. FlashDecoding++: Faster Large Language Model Inference with Asynchronization, Flat GEMM Optimization, and Heuristics. In *Proceedings of Machine Learning and Systems*, P. Gibbons, G. Pekhimenko, and C. De Sa (Eds.), Vol. 6. 148–161. https://proceedings.mlsys.org/paper_files/paper/2024/file/5321b1dabcd2be188d796c21b733e8c7-Paper-Conference.pdf
- [35] Cunhen Hu, Heyang Huang, Liangliang Xu, Xusheng Chen, Jiang Xu, Shuang Chen, Hao Feng, Chenxi Wang, Sa Wang, Yungang Bao, Ninghui Sun, and Yizhou Shan. 2024. Inference without Interference: Disaggregate LLM Inference for Mixed Downstream Workloads. arXiv:2401.11181 [cs.DC] <https://arxiv.org/abs/2401.11181>
- [36] Haiyang Huang, Newsha Ardalani, Anna Sun, Liu Ke, Shruti Bhosale, Hsien-Hsin S. Lee, Carole-Jean Wu, and Benjamin Lee. 2024. Toward Efficient Inference for Mixture of Experts. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*. <https://openreview.net/forum?id=stXtBqyTWX>
- [37] Abhinav Jangda, Jun Huang, Guodong Liu, Amir Hossein Nodehi Sabet, Saeed Maleki, Youshan Miao, Madanlal Musuvathi, Todd Mytkowicz,

- and Olli Saarikivi. 2022. Breaking the Computation and Communication Abstraction Barrier in Distributed Machine Learning Workloads. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems* (Lausanne, Switzerland) (ASPLOS '22). Association for Computing Machinery, New York, NY, USA, 402–416. <https://doi.org/10.1145/3503222.3507778>
- [38] Abhinav Jangda, Saeed Maleki, Maryam Mehri Dehnavi, Madan Musuvathi, and Olli Saarikivi. 2024. A Framework for Fine-Grained Synchronization of Dependent GPU Kernels. In *Proceedings of the 2024 IEEE/ACM International Symposium on Code Generation and Optimization* (Edinburgh, United Kingdom) (CGO '24). IEEE Press, 93–105. <https://doi.org/10.1109/CGO57630.2024.10444873>
- [39] Hao Kang, Srikant Bharadwaj, James Hensman, Tushar Krishna, Victor Rühle, and Saravan Rajmohan. 2024. TurboAttention: Efficient Attention Approximation For High Throughputs LLMs. arXiv:2412.08585 [cs.LG] <https://arxiv.org/abs/2412.08585>
- [40] Scott J. Krieder, Justin M. Wozniak, Timothy Armstrong, Michael Wilde, Daniel S. Katz, Benjamin Grimmer, Ian T. Foster, and Ioan Raicu. 2014. Design and evaluation of the gemtc framework for GPU-enabled many-task computing. In *Proceedings of the 23rd International Symposium on High-Performance Parallel and Distributed Computing* (Vancouver, BC, Canada) (HPDC '14). Association for Computing Machinery, New York, NY, USA, 153–164. <https://doi.org/10.1145/2600212.2600228>
- [41] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles* (Koblenz, Germany) (SOSP '23). Association for Computing Machinery, New York, NY, USA, 611–626. <https://doi.org/10.1145/3600006.3613165>
- [42] Ao Li, Bojian Zheng, Gennady Pekhimenko, and Fan Long. 2022. Automatic Horizontal Fusion for GPU Kernels. In *2022 IEEE/ACM International Symposium on Code Generation and Optimization* (CGO). 14–27. <https://doi.org/10.1109/CGO53902.2022.9741270>
- [43] Yun Liang, Huynh Phung Huynh, Kyle Rupnow, Rick Siow Mong Goh, and Deming Chen. 2015. Efficient GPU Spatial-Temporal Multitasking. *IEEE Transactions on Parallel and Distributed Systems* 26, 3 (2015), 748–760. <https://doi.org/10.1109/TPDS.2014.2313342>
- [44] Muhammad Osama, Duane Merrill, Cris Cecka, Michael Garland, and John D. Owens. 2023. Stream-K: Work-Centric Parallel Decomposition for Dense Matrix-Matrix Multiplication on the GPU. In *Proceedings of the 28th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming* (Montreal, QC, Canada) (PPoPP '23). Association for Computing Machinery, New York, NY, USA, 429–431. <https://doi.org/10.1145/3572848.3577479>
- [45] Sreepathi Pai, Matthew J. Thazhuthaveetil, and R. Govindarajan. 2013. Improving GPGPU concurrency with elastic kernels. In *Proceedings of the Eighteenth International Conference on Architectural Support for Programming Languages and Operating Systems* (Houston, Texas, USA) (ASPLOS '13). Association for Computing Machinery, New York, NY, USA, 407–418. <https://doi.org/10.1145/2451116.2451160>
- [46] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture* (ISCA). 118–132. <https://doi.org/10.1109/ISCA59077.2024.00019>
- [47] Ramya Prabhu, Ajay Nayak, Jayashree Mohan, Ramachandran Ramjee, and Ashish Panwar. 2025. vAttention: Dynamic Memory Management for Serving LLMs without PagedAttention. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1* (Rotterdam, Netherlands) (ASPLOS '25). Association for Computing Machinery, New York, NY, USA, 1133–1150. <https://doi.org/10.1145/3669940.3707256>
- [48] Rya Sanovar, Srikant Bharadwaj, Renee St. Amant, Victor Rühle, and Saravan Rajmohan. 2024. Lean Attention: Hardware-Aware Scalable Attention Mechanism for the Decode-Phase of Transformers. arXiv:2405.10480 [cs.AR] <https://arxiv.org/abs/2405.10480>
- [49] Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. 2024. FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-precision. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*. <https://openreview.net/forum?id=tVConYid20>
- [50] Ying Sheng, Shiyi Cao, Dacheng Li, Banghua Zhu, Zhuohan Li, Danyang Zhuo, Joseph E. Gonzalez, and Ion Stoica. 2024. Fairness in Serving Large Language Models. In *18th USENIX Symposium on Operating Systems Design and Implementation* (OSDI 24). USENIX Association, Santa Clara, CA, 965–988. <https://www.usenix.org/conference/osdi24/presentation/sheng>
- [51] Yixin Song, Zeyu Mi, Haotong Xie, and Haibo Chen. 2024. PowerInfer: Fast Large Language Model Serving with a Consumer-grade GPU. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles* (Austin, TX, USA) (SOSP '24). Association for Computing Machinery, New York, NY, USA, 590–606. <https://doi.org/10.1145/3694715.3695964>
- [52] Jovan Stojkovic, Chaojie Zhang, Íñigo Goiri, Josep Torrellas, and Esha Choukse. 2024. DynamoLLM: Designing LLM Inference Clusters for Performance and Energy Efficiency. arXiv:2408.00741 [cs.AI] <https://arxiv.org/abs/2408.00741>
- [53] Mohamed Wahib and Naoya Maruyama. 2014. Scalable Kernel Fusion for Memory-Bound GPU Applications. In *SC '14: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 191–202. <https://doi.org/10.1109/SC.2014.21>
- [54] Shibo Wang, Jinliang Wei, Amit Sabne, Andy Davis, Berkin Ilbeyi, Blake Hechtman, Dehao Chen, Karthik Srinivasa Murthy, Marcello Maggioni, Qiao Zhang, Sameer Kumar, Tongfei Guo, Yuanzhong Xu, and Zongwei Zhou. 2022. Overlap Communication with Dependent Computation via Decomposition in Large Deep Learning Models. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1* (Vancouver, BC, Canada) (ASPLOS 2023). Association for Computing Machinery, New York, NY, USA, 93–106. <https://doi.org/10.1145/3567955.3567959>
- [55] Zhenning Wang, Jun Yang, Rami Melhem, Bruce Childers, Youtao Zhang, and Minyi Guo. 2016. Simultaneous Multikernel GPU: Multitasking throughput processors via fine-grained sharing. In *2016 IEEE International Symposium on High Performance Computer Architecture* (HPCA). 358–369. <https://doi.org/10.1109/HPCA.2016.7446078>
- [56] Bo Wu, Guoyang Chen, Dong Li, Xipeng Shen, and Jeffrey Vetter. 2015. Enabling and Exploiting Flexible Task Assignment on GPU through SM-Centric Program Transformations. In *Proceedings of the 29th ACM on International Conference on Supercomputing* (Newport Beach, California, USA) (ICS '15). Association for Computing Machinery, New York, NY, USA, 119–130. <https://doi.org/10.1145/2751205.2751213>
- [57] Bingyang Wu, Shengyu Liu, Yinmin Zhong, Peng Sun, Xuanzhe Liu, and Xin Jin. 2024. LoongServe: Efficiently Serving Long-Context Large Language Models with Elastic Sequence Parallelism. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles* (Austin, TX, USA) (SOSP '24). Association for Computing Machinery, New York, NY, USA, 640–654. <https://doi.org/10.1145/3694715.3695948>
- [58] Bingyang Wu, Yinmin Zhong, Zili Zhang, Gang Huang, Xuanzhe Liu, and Xin Jin. 2023. Fast Distributed Inference Serving for Large Language Models. arXiv:2305.05920 [cs.LG] <https://arxiv.org/abs/2305.05920>
- [59] Haicheng Wu, Gregory Diamos, Srihari Cadambi, and Sudhakar Yalamanchili. 2012. Kernel Weaver: Automatically Fusing Database Primitives for Efficient GPU Computation. In *2012 45th Annual IEEE/ACM International Symposium on Microarchitecture*. 107–118. <https://doi.org/10.1109/MICRO.2012.19>

- [60] Zihao Ye, Lequn Chen, Ruihang Lai, Wuwei Lin, Yineng Zhang, Stephanie Wang, Tianqi Chen, Baris Kasikci, Vinod Grover, Arvind Krishnamurthy, and Luis Ceze. 2025. FlashInfer: Efficient and Customizable Attention Engine for LLM Inference Serving. arXiv:2501.01005 [cs.DC] <https://arxiv.org/abs/2501.01005>
- [61] Tsung Tai Yeh, Amit Sabne, Putt Sakdhnagool, Rudolf Eigenmann, and Timothy G. Rogers. 2017. Pagoda: Fine-Grained GPU Resource Virtualization for Narrow Tasks. In *Proceedings of the 22nd ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming* (Austin, Texas, USA) (PPoPP '17). Association for Computing Machinery, New York, NY, USA, 221–234. <https://doi.org/10.1145/3018743.3018754>
- [62] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A Distributed Serving System for Transformer-Based Generative Models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. USENIX Association, Carlsbad, CA, 521–538. <https://www.usenix.org/conference/osdi22/presentation/yu>
- [63] Han Zhao, Weihao Cui, Quan Chen, and Minyi Guo. 2023. ISPA: Exploiting Intra-SM Parallelism in GPUs via Fine-Grained Resource Management. *IEEE Trans. Comput.* 72, 5 (2023), 1473–1487. <https://doi.org/10.1109/TC.2022.3214088>
- [64] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. 2024. SGLang: Efficient Execution of Structured Language Model Programs. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*. <https://openreview.net/forum?id=VqkAKQibpq>
- [65] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. USENIX Association, Santa Clara, CA, 193–210. <https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin>
- [66] Kan Zhu, Yilong Zhao, Liangyu Zhao, Gefei Zuo, Yile Gu, Dedong Xie, Yufei Gao, Qinyu Xu, Tian Tang, Zihao Ye, Keisuke Kamahori, Chien-Yu Lin, Stephanie Wang, Arvind Krishnamurthy, and Baris Kasikci. 2024. NanoFlow: Towards Optimal Large Language Model Serving Throughput. arXiv:2408.12757 [cs.DC] <https://arxiv.org/abs/2408.12757>